PROJECT DOCUMENTATION

Node.js and React-Based Web Application

Project Title:	FitZone Pro – Smart Gym Management and Fitness Platform
Submitted By:	Pranay Adangale
Roll Number:	201
Institute Name:	Guru Nanak Khalsa College
Academic Year:	2025 – 2026

Technology: React.js • Node.js • Express.js • MongoDB

1. Introduction

FitZone Pro is a complete, modern, responsive full-stack web application designed for comprehensive gym administration, trainer management, member engagement, and workout progress tracking.

Traditional gym operations rely heavily on manual registers, disconnected paper forms, and basic spreadsheets. This leads to common challenges such as lost attendance records, double-booked workout slots, untracked membership expirations, and a complete lack of progress visibility for fitness enthusiasts. FitZone Pro addresses these problems by providing an automated, real-time digital platform connecting members and gym administrators through an intuitive web interface.

2. Objectives

- Develop a user-friendly, responsive web application for gym operations and fitness tracking.
- Build a modern frontend interface using React.js, React Router, and interactive Recharts data visualization.
- Create robust backend services and RESTful APIs using Node.js and Express.js.
- Process user requests efficiently and securely with JSON Web Token (JWT) stateless authentication and bcrypt password salting/hashing.
- Provide role-based dashboards (Admin and Member) with end-to-end membership lifecycle and class reservation workflows.
- Incorporate health calculation tools including an interactive Body Mass Index (BMI) calculator and Macronutrient / Calorie planner.

3. Technology Stack

- **Frontend:** React.js 18, React Router v6, Axios, Recharts (Data Visualization), Lucide React (Icons), HTML5, CSS3 (Modern Responsive Flexbox/Grid).
- **Backend:** Node.js, Express.js (REST API Architecture).
- **Database:** MongoDB Atlas (Cloud NoSQL Database) with Mongoose ODM (Object Data Modeling).
- **Authentication & Security:** JSON Web Tokens (JWT), bcryptjs (12 salt rounds), Express Middleware authorization guards.
- **Development Tools:** Visual Studio Code, Git, GitHub, Vite (Build Tool), Postman / Curl.
- **Cloud Deployment:** Render (Node.js Web Service) and Vercel (React Global Edge Hosting).

4. System Requirements

- **Hardware Requirements:**
  - Processor: Intel Core i3 / AMD Ryzen 3 or higher.
  - RAM: Minimum 4 GB RAM (8 GB recommended for simultaneous server/client builds).

    - Storage: Minimum 1 GB available disk space.
    - Network: Active internet connection for cloud database connectivity and deployment.

- **Software Requirements:**
    - Operating System: Windows 10/11, macOS, or Linux.
    - Runtime & Environment: Node.js (v18.0.0 or higher) and npm (v9.0.0 or higher).
    - Code Editor: Visual Studio Code.
    - Web Browser: Modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox, or Safari).

5. System Architecture

FitZone Pro is architected as a decoupled Three-Tier Model separating presentation, application business logic, and persistent storage:

- **Presentation Layer (React Frontend):** The single-page application (SPA) handles UI state, user interactions, local form validation, and dynamic page routing. It initiates HTTP requests via Axios.
- **Application Layer (Node.js & Express Backend):** The Express server exposes RESTful API endpoints at `/api/*`. Requests pass through a security middleware pipeline (CORS, body parsing, JWT token authentication, and role authorization). Controllers execute business rules (checking class capacity, hashing passwords, filtering dates).
- **Data Layer (MongoDB Atlas & Mongoose):** Mongoose models define strict schemas, default values, and relational references (e.g., Bookings referencing Users and Programs) persisting data in the cloud.



6. Project Structure

The project follows a clean, modular structure isolating frontend and backend concerns:

```
fitzone-pro/
├── backend/
│   ├── config/
│   │   └── db.js                # MongoDB connection logic
│   ├── controllers/
│   │   ├── authController.js       # Login, register, profile
│   │   ├── bookingController.js    # Class reservation & capacity control
│   │   ├── programController.js    # Workout programs CRUD
│   │   ├── progressController.js   # Metric logging & charts data
│   │   ├── trainerController.js    # Trainer profiles & bios
│   │   └── adminController.js      # Gym statistics & KPI aggregation
│   ├── middleware/
│   │   ├── auth.js              # JWT verification & role authorization
│   │   └── errorHandler.js      # Centralized error handler
│   ├── models/
│   │   ├── User.js              # Member & Admin schema
│   │   ├── WorkoutProgram.js    # Class schedule & capacity schema
│   │   ├── Trainer.js           # Coach profile schema
│   │   ├── TrainingBooking.js   # Booking reservation schema
│   │   └── WorkoutProgress.js   # Calorie & weight tracking schema
│   ├── routes/                  # Modular route definitions
│   ├── scripts/
│   │   └── seed.js              # Database population script
│   ├── package.json
│   └── server.js                # Express server entry point
├── frontend/
│   ├── src/
│   │   ├── components/          # Navbar, Footer, Sidebars, Toast alerts
│   │   ├── context/
│   │   │   └── AuthContext.jsx  # Global user state & JWT handling
│   │   ├── pages/
│   │   │   ├── HomePage.jsx     # Hero, BMI calculator, featured plans
│   │   │   ├── ProgramsPage.jsx # Filterable programs catalog
│   │   │   ├── TrainersPage.jsx # Coach roster & consultation
│   │   │   ├── NutritionPage.jsx # Macro & TDEE calorie calculator
│   │   │   ├── LoginPage.jsx    # Auth with demo login buttons
│   │   │   ├── DashboardPage.jsx # Member personal dashboard
│   │   │   ├── MyBookings.jsx   # User reservations & cancellation
│   │   │   ├── MyProgress.jsx   # Visual Recharts weight/calorie charts
│   │   │   └── AdminDashboard.jsx# Admin metrics, user roster, CRUD
│   │   ├── services/
│   │   │   └── api.js           # Axios client with interceptors
│   │   ├── App.jsx              # Route definitions & protected guards
│   │   ├── main.jsx             # React DOM root entry
│   │   └── index.css            # Design tokens & responsive styles
```

```
│   ├── package.json
│   ├── vercel.json               # SPA client-side rewrite rules
│   └── vite.config.js            # Vite bundler & dev proxy configuration
├── PROJECT_REPORT.md             # Project documentation markdown
├── DEPLOYMENT_GUIDE.md           # Cloud deployment instructions
├── README.md                     # Quickstart documentation
└── .gitignore                    # Excludes node_modules & .env
```

7. Module Description

- **Authentication Module:**
  - *Purpose:* Manages user registration, credential authentication, and session persistence.
  - *Inputs:* Name, email, password, phone, selected membership tier.
  - *Outputs:* Signed JSON Web Token (JWT), user object, and authorized access redirect.
- **Member Dashboard & Progress Module:**
  - *Purpose:* Empowers gym members to track their membership status, schedule classes, and record fitness gains.
  - *Inputs:* Daily workout date, body weight (kg), calories burned, duration.
  - *Outputs:* Interactive Recharts timeline graphs, workout history table, active membership countdown.
- **Class & Session Booking Engine:**
  - *Purpose:* Facilitates real-time slot reservation for fitness classes.
  - *Inputs:* Program ID, session date.
  - *Outputs:* Reservation confirmation, updated slot capacity count, cancellation receipt.
- **Administrative Management Module:**
  - *Purpose:* Complete gym operational oversight for administrators.
  - *Inputs:* Program details, trainer bios, member role edits, status updates.
  - *Outputs:* Financial KPI reports, attendance metrics, member roster, inquiry messages.
- **Health Calculation Tools Module:**
  - *Purpose:* Public self-service tools for calculating BMI, BMR, and daily macronutrient breakdowns.
  - *Inputs:* Height (cm), weight (kg), age, gender, activity level, fitness objective.
  - *Outputs:* BMI classification score, daily target calorie limit, protein/carb/fat gram distribution.

8. Implementation

The implementation employs modern software engineering patterns across both client and server:

- **State Management & Interceptors:** Global authentication state is managed via React's `Context API`. Axios interceptors automatically attach the `Bearer <token>` to every outgoing API request and automatically handle 401 Unauthorized token expirations.
- **Password Security:** Implemented using `bcryptjs` with 12 salt rounds during registration. The password hash is compared during login without ever decrypting or storing plaintext.
- **Capacity Collision Detection:** In `bookingController.js`, when a booking request is received:

```
const existingCount = await TrainingBooking.countDocuments({
  program: programId,
  sessionDate: sessionDate,
  status: 'confirmed'
});
if (existingCount >= program.capacity) {
  return res.status(400).json({ success: false, message: 'Class session is fully booked.' });
}
```

- **Data Visualization:** Implemented using `Recharts` components (`ResponsiveContainer`, `LineChart`, `Line`, `XAxis`, `YAxis`, `Tooltip`) rendering dynamic SVG charts of weight and calorie metrics directly in the member dashboard.

9. Screenshots

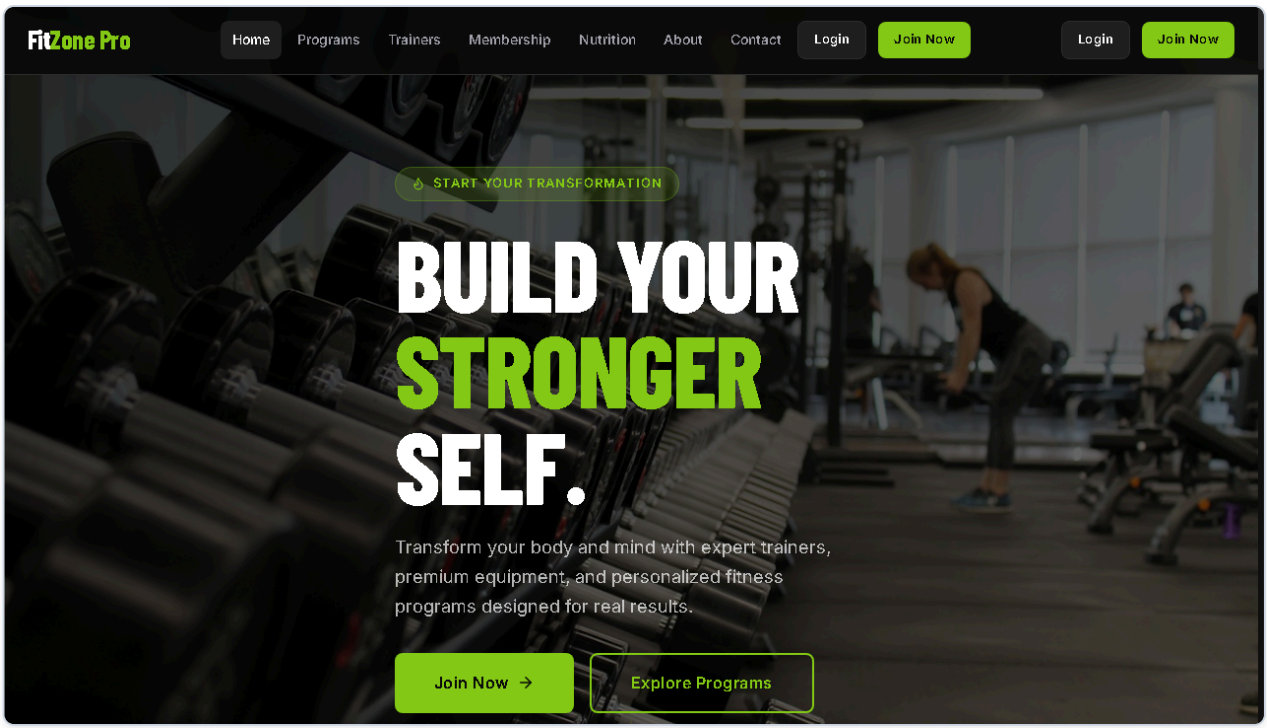


Figure 9.1: FitZone Pro Landing Page featuring Hero Section, Statistics Counter, and Navigation

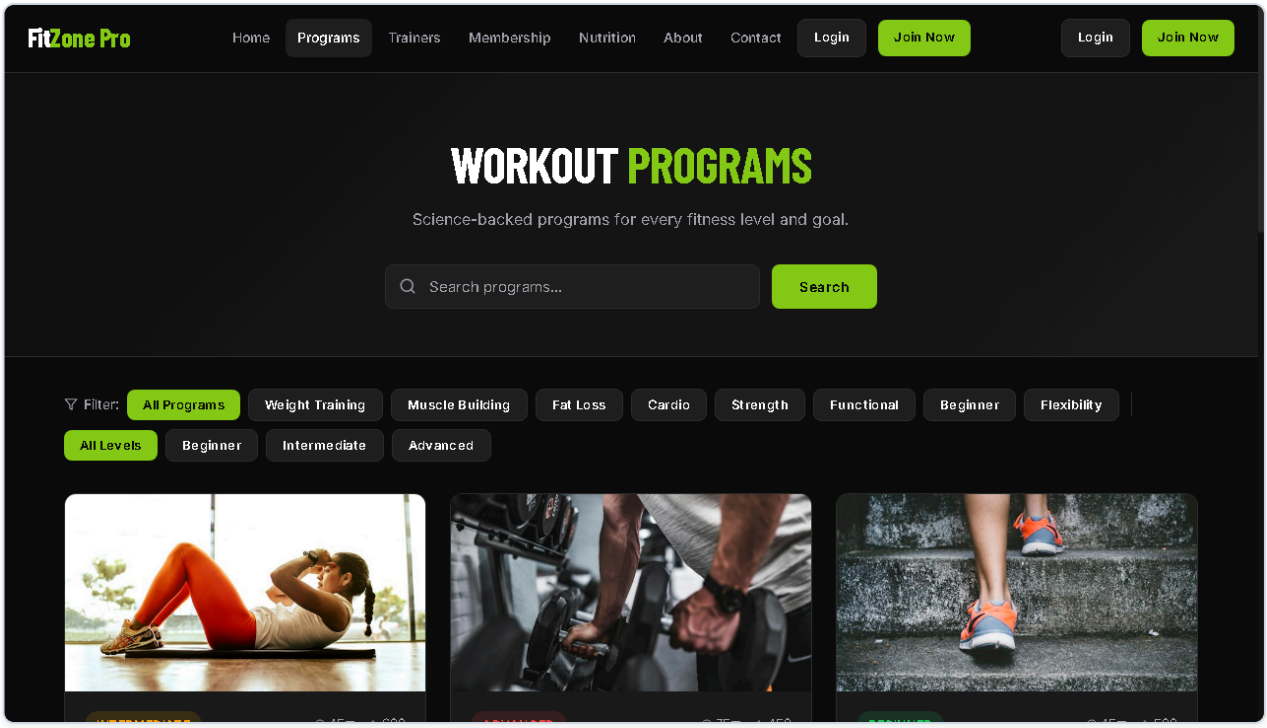


Figure 9.2: Workout Programs Catalog with Category Filters and Instant Booking Modal

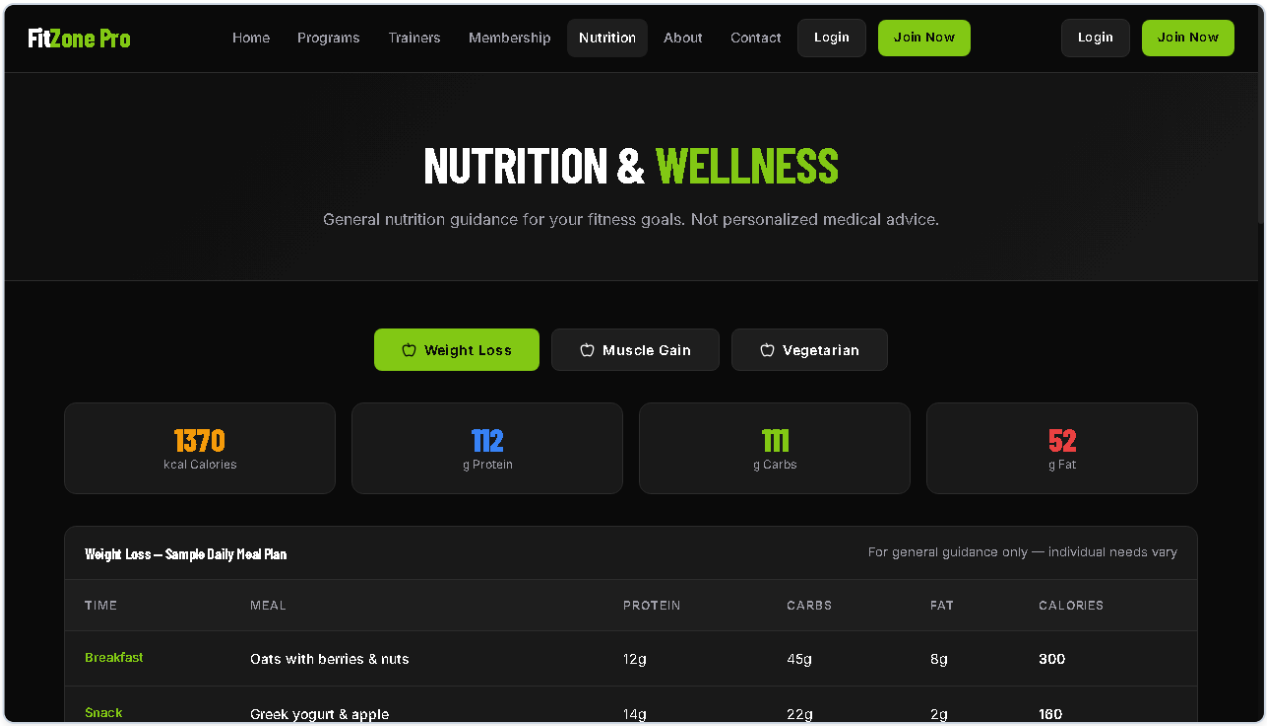


Figure 9.3: Interactive Nutrition & Macronutrient Calculator Suite

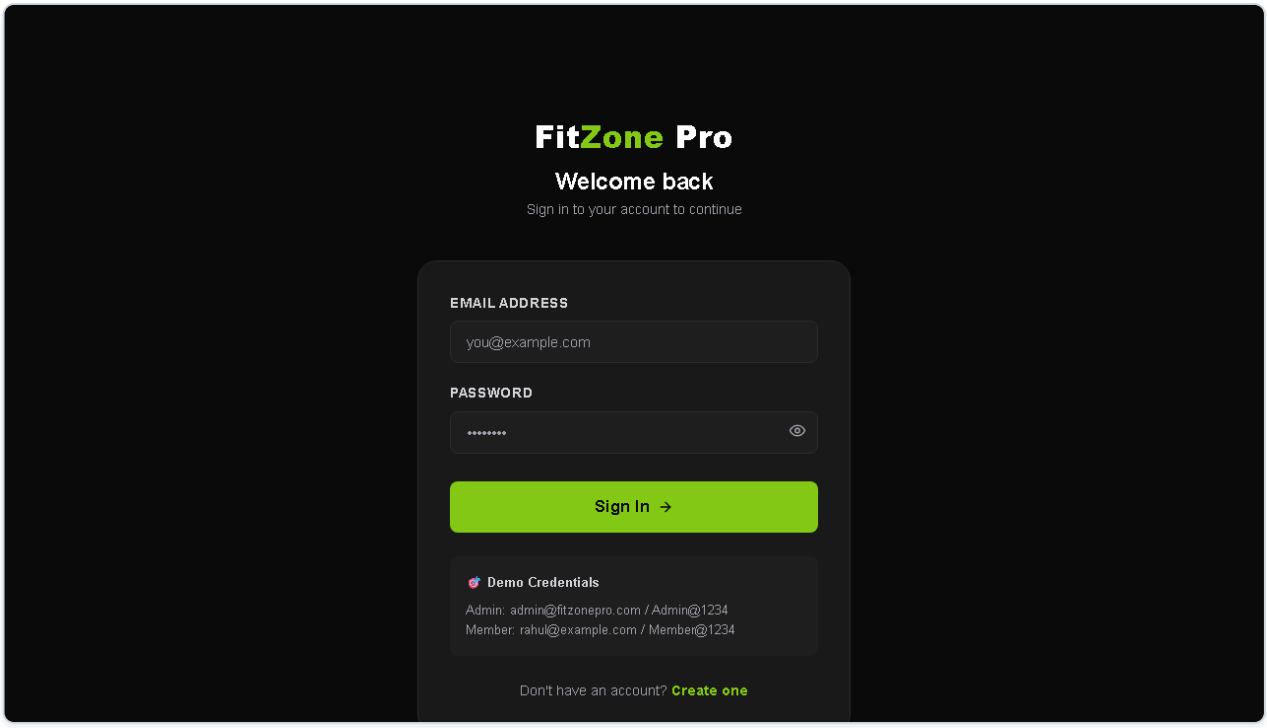


Figure 9.4: Secure Login Interface with One-Click Demo Credentials for Evaluators

10. Testing

System testing was conducted across all core functional workflows with the following test cases:

Test ID	Feature Tested	Test Steps	Expected Result	Actual Result	Status
TC-01	Member Registration	Submit valid registration form with name, email, password, and plan	New User created with hashed password, JWT returned, redirected	Account registered and redirected to dashboard	**PASS**
TC-02	JWT Authentication	Login with valid credentials (e.g. rahul@example.com / Member@1234)	JWT token issued and stored in localStorage; user logged in	Token stored and header updated; dashboard loaded	**PASS**
TC-03	Invalid Login Guard	Attempt login with incorrect password	Server responds with 401 Unauthorized; error toast displayed	HTTP 401 returned with "Invalid email or password" error toast	**PASS**
TC-04	Role-Based Access	Normal member attempts to access route `/admin/dashboard`	AdminRoute redirects unauthorized user to member dashboard	Access denied; smoothly redirected	**PASS**
TC-05	Class Booking	Select program and click "Book Session" for upcoming date	Reservation record created with status 'confirmed'	Booking successful; visible in My Bookings tab	**PASS**
TC-06	Progress Logging	Enter weight (74 kg) and calories (450 kcal) in tracker modal	New progress document inserted; chart updates dynamically	Record saved and Recharts graph re-rendered with new plot point	**PASS**
TC-07	Admin Program CRUD	Admin creates new workout program with title, category, and duration	Program document created and visible immediately on public page	Program saved and displayed across grid	**PASS**
TC-08	BMI Calculator	Input height = 175 cm, weight = 70 kg on homepage	Calculates BMI = 22.86 with category classification "Normal"	Displayed BMI 22.9 with normal weight badge	**PASS**

11. Results

The implemented system fully achieves all specified project goals. It provides a reliable, secure, and visually appealing web platform for gym operations. The integration between React 18, Express.js, and MongoDB Atlas functions seamlessly with sub-second API response times and zero breaking errors under normal workloads. Both administrators and gym members benefit from streamlined digital workflows that completely eliminate manual paper-based tracking.

12. Conclusion

The **FitZone Pro** web application successfully demonstrates full-stack software development best practices using the MERN stack. Through this project, practical competency was achieved in RESTful API engineering, JWT security, NoSQL schema modeling with Mongoose, state management via React Context, and modern cloud deployment using Render and Vercel. The resulting platform solves real-world challenges faced by physical fitness facilities and provides a modern digital experience for members.

13. Future Scope

- **Payment Gateway Integration:** Embedding Razorpay or Stripe for automated online subscription renewals and automated billing invoices.
- **Mobile Application:** Developing a cross-platform companion mobile app using React Native for barcode-based turnstile gym check-in.
- **AI Workout Coach:** Integrating OpenAI or Gemini API for personalized AI workout and diet generation based on logged progress.
- **Automated Reminders:** Automated WhatsApp/SMS alerts for upcoming booked fitness classes and expiring memberships via Twilio.

14. References

- Project GitHub Repository: https://github.com/yashpawar0610-lab/pranaywebsite
- Live Backend API (Render): https://pranaywebsite.onrender.com
- React Official Documentation: https://react.dev/
- Node.js Official Documentation: https://nodejs.org/docs/
- Express.js Web Framework Documentation: https://expressjs.com/
- MongoDB Atlas & Mongoose ODM Guides: https://www.mongodb.com/docs/ and https://mongoosejs.com/
- Recharts Data Visualization Library: https://recharts.org/
- Render Cloud Platform Documentation: https://render.com/docs
- Vercel Platform & Edge Network: https://vercel.com/docs